

Understanding and Evaluating BLUE: An Active Queue Management Algorithm

Shatabdi Dutta Chowdhury
Student ID - 2105123
Section - C

February 7, 2026



The Problem – Why This Research Matters?



Skyrocketing Internet Traffic :

Global data demands are outpacing hardware capabilities.

Buffer Limitations:

Routers have physical memory limits; they cannot hold "infinite" data.

The Triple Threat of Congestion:

- Packet Loss : Data disappears, requiring retransmission.
- High Latency/Delay : Significant lag in data delivery.
- Jitter : Inconsistent timing between packets, ruining real-time communication (like VoIP or Gaming).

The Critical Consequences

Resource Wastage: When a packet drops , resources to send that packet (bandwidth, power, time) are wasted.

The Downward Spiral: High packet loss rates lead to constant re-sending, which further chokes the network.

Congestion Collapse: This inefficiency can lead to a total network shutdown where no useful data gets through.



Understanding Network Congestion

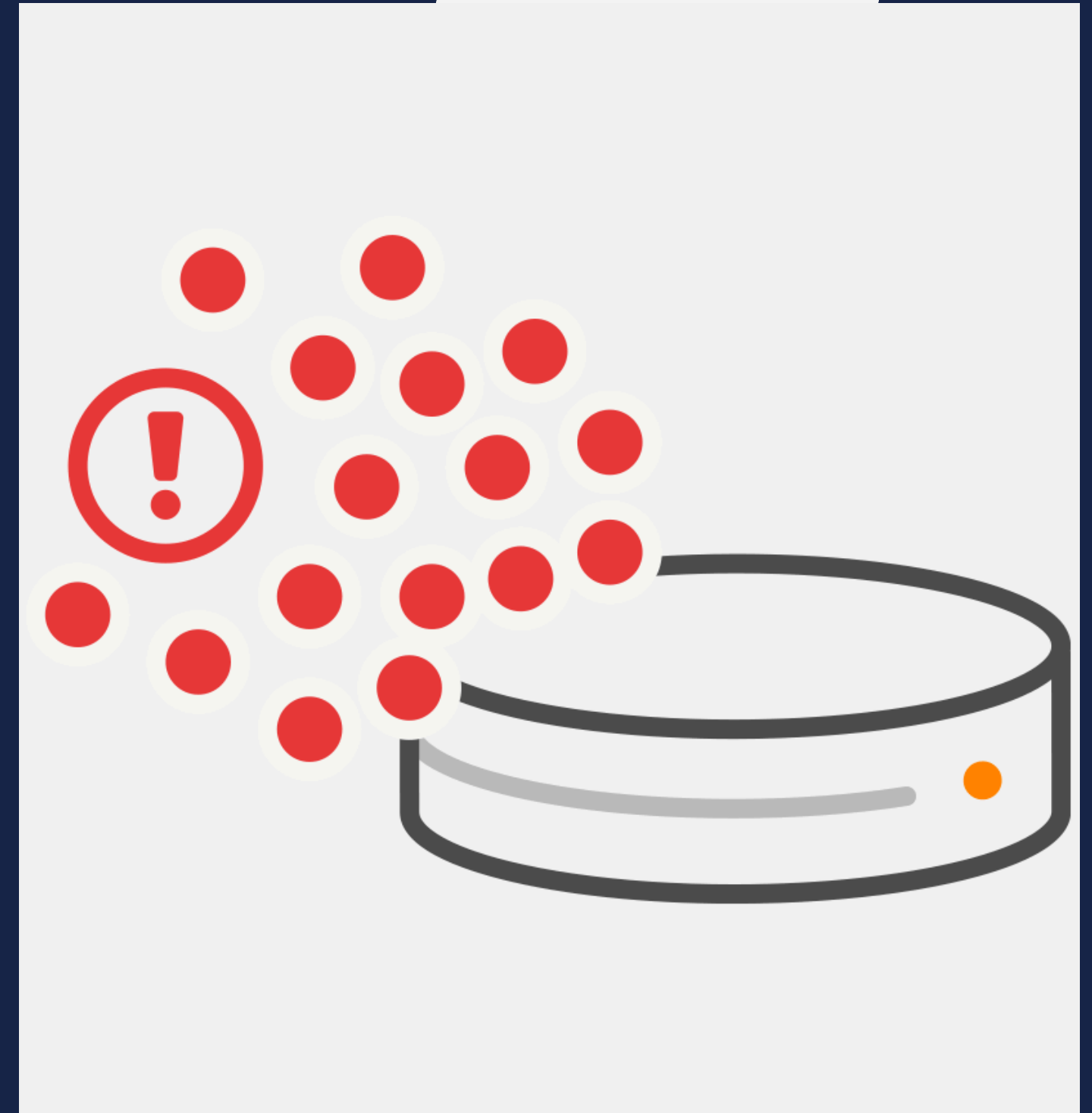
What is Congestion ?

Definition:

A state in which the number of packets being transmitted through the network approaches or exceeds the packet-handling capacity of the network nodes (routers).

The Chain Reaction:

1. **Queue Accumulation:** Incoming data exceeds outgoing capacity, causing the router's buffer (queue) to fill up.
2. **Increased Latency:** Packets are forced to wait in the queue, causing significant delays.
3. **Packet Dropping:** Once the buffer is 100% full, the router has no choice but to discard incoming packets (Tail Drop).



Traditional Solution — The DropTail Mechanism

How DropTail Works

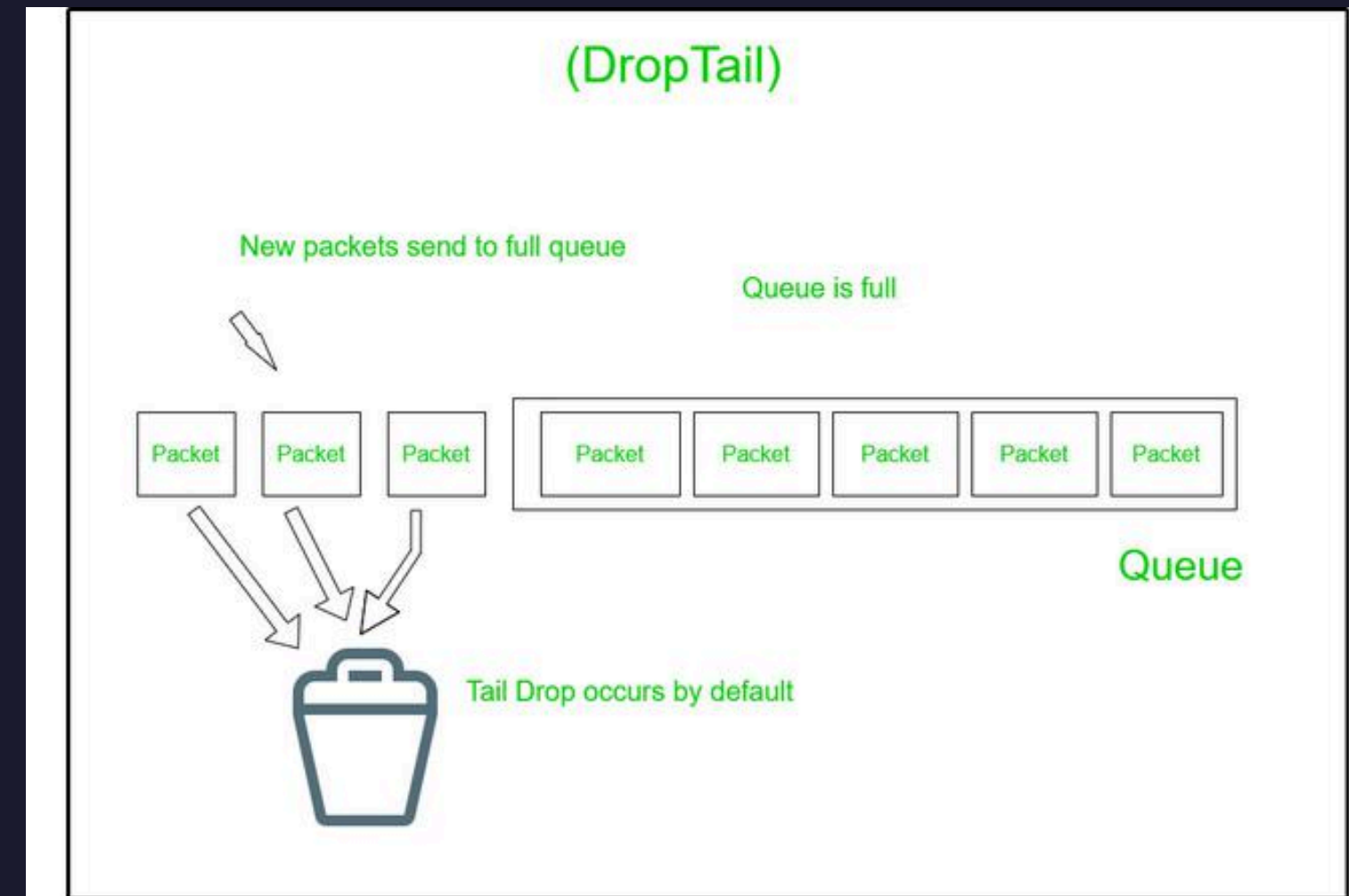
- **Passive Management:** Packets are accepted until the maximum buffer capacity is reached.
- **Full Buffer Drops:** New incoming packets are only discarded after the queue is 100% full.

- **Key Drawbacks:**

Delayed Detection: It ignores congestion until the system is already overwhelmed.

Sudden Packet Loss: Leads to "bursty" drops rather than smooth traffic management.

TCP Global Synchronization: Multiple senders reduce their speed at the exact same time, causing significant network underutilization.



RED (Random Early Detection) — Proactive Management

The Proactive Approach

- **Average Queue Length:** Unlike DropTail, RED monitors the average queue size over time to detect early signs of congestion.
- **Probabilistic Dropping:** It doesn't wait for a full buffer. It starts dropping packets randomly before the queue is full to signal senders to slow down.
- **Dual Thresholds:**
 - Below `min_th`: No packets are dropped (Safe zone).
 - Between `min_th` & `max_th`: Packets are dropped with an increasing probability.
 - Above `max_th`: All packets are dropped (Congestion zone).
- **Goal:** To maintain a small queue size and avoid "Global Synchronization."

Red Scenerio

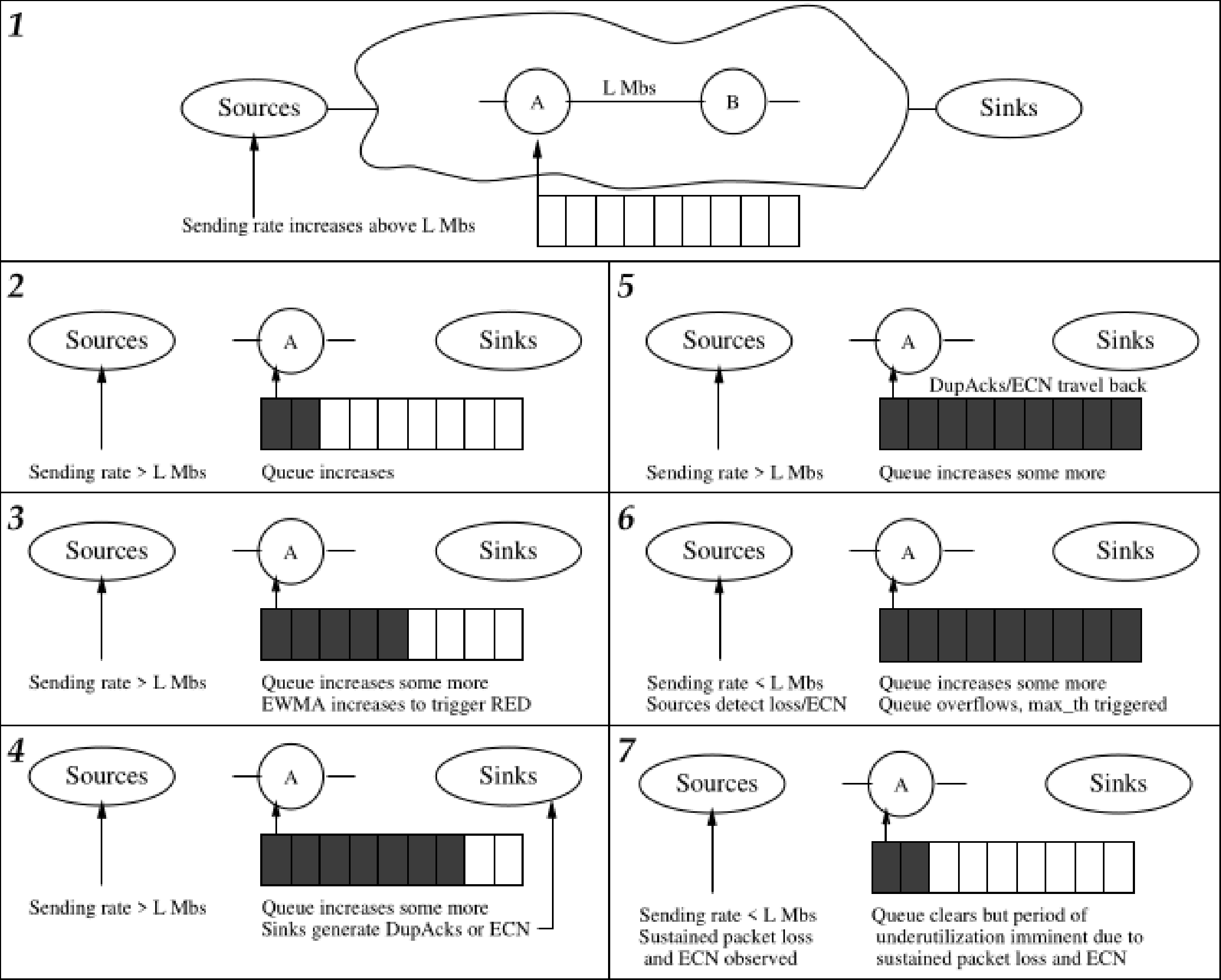
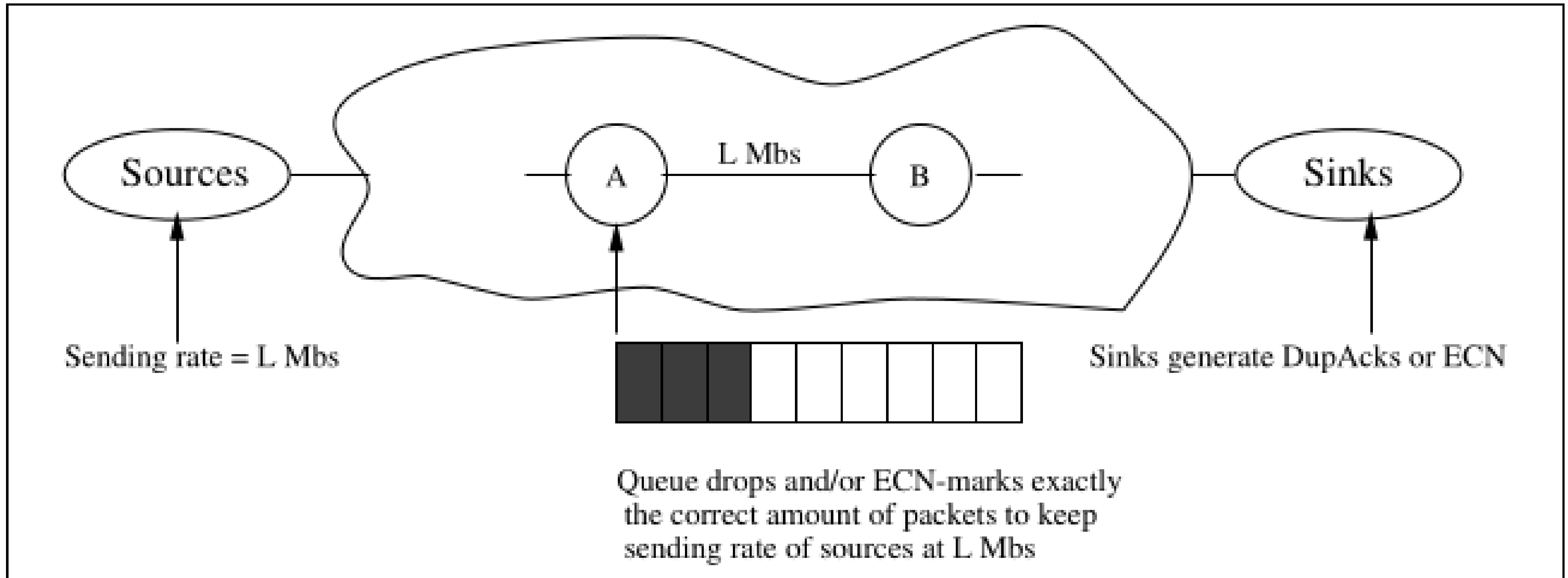


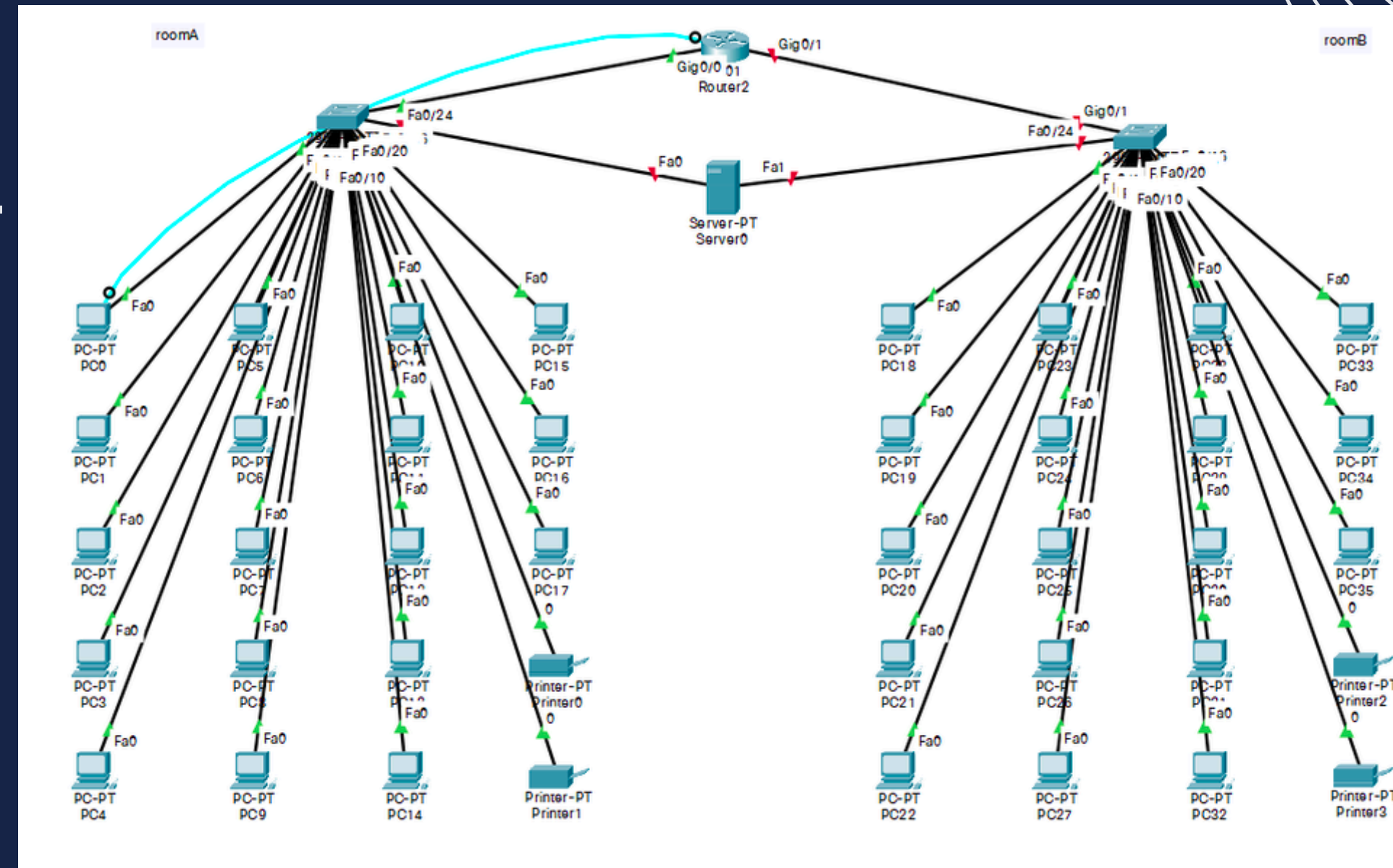
Figure 1: RED example

An Ideal Scenario



The Limitations of RED (Why It Fails)

- **Inaccurate Congestion Metric:** Average queue length is not always a reliable indicator of true congestion.
- **Parameter Sensitivity:** RED requires meticulous tuning of multiple parameters (minth, maxth, wq). If not tuned perfectly, performance drops significantly.
- **Bursty Traffic Instability:** Under high-speed or "bursty" traffic, RED becomes unstable, leading to either underutilization or excessive drops.
- **The Buffer Dilemma:** To work effectively, RED often requires large buffers, which ironically introduces High Delay and Jitter (Bufferbloat).



The Key Idea Behind BLUE

A Paradigm Shift in AQM

Beyond Queue Length:

Unlike RED, BLUE completely ignores queue length as a primary congestion metric.

Experience-Based Feedback:

The algorithm manages congestion based on actual events occurring at the router's interface.

Event-Driven Observation:

- Packet Loss (Congestion):

If the buffer overflows, BLUE realizes the drop probability is too low and increases it.

- Link Idle Time (Underutilization):

If the link is idle while the queue is empty, BLUE realizes the drop probability is too high and decreases it.

How BLUE Works (Algorithm)

- Event 1: Packet Loss (or Queue Overflow)

If the router loses a packet or the queue exceeds a set limit (L):

Action: Increase p_m by a small constant ($d1$).

Reason: The network is congested; we need to drop/mark more packets to slow down the senders.

- Event 2: Link Idle

If the transmission link becomes idle (no data to send):

Action: Decrease p_m by a small constant ($d2$).

Reason: The link is underutilized; we should drop/mark fewer packets to allow more traffic.

- The "Freeze_Time" Variable:

To prevent the algorithm from reacting too fast or becoming unstable, it waits for a specific duration (`freeze_time`) before making the next adjustment.

Upon packet loss (or $Q_{len} > L$) event:

if ($(now - last_update) > freeze_time$) then

$p_m = p_m + d1$

$last_update = now$

Upon link idle event:

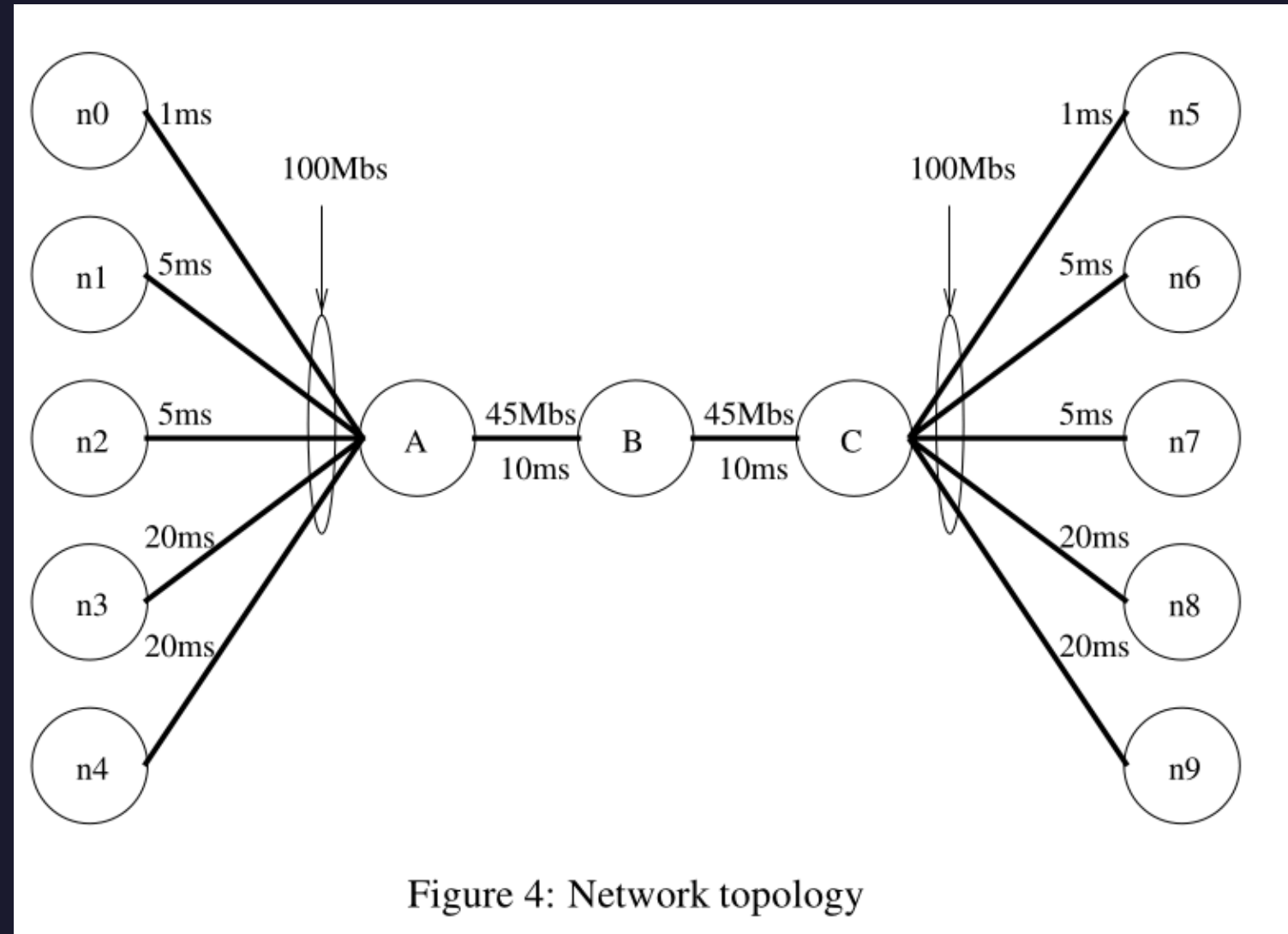
if ($(now - last_update) > freeze_time$) then

$p_m = p_m - d2$

$last_update = now$

Figure 3: The BLUE algorithm

Experimental Setup



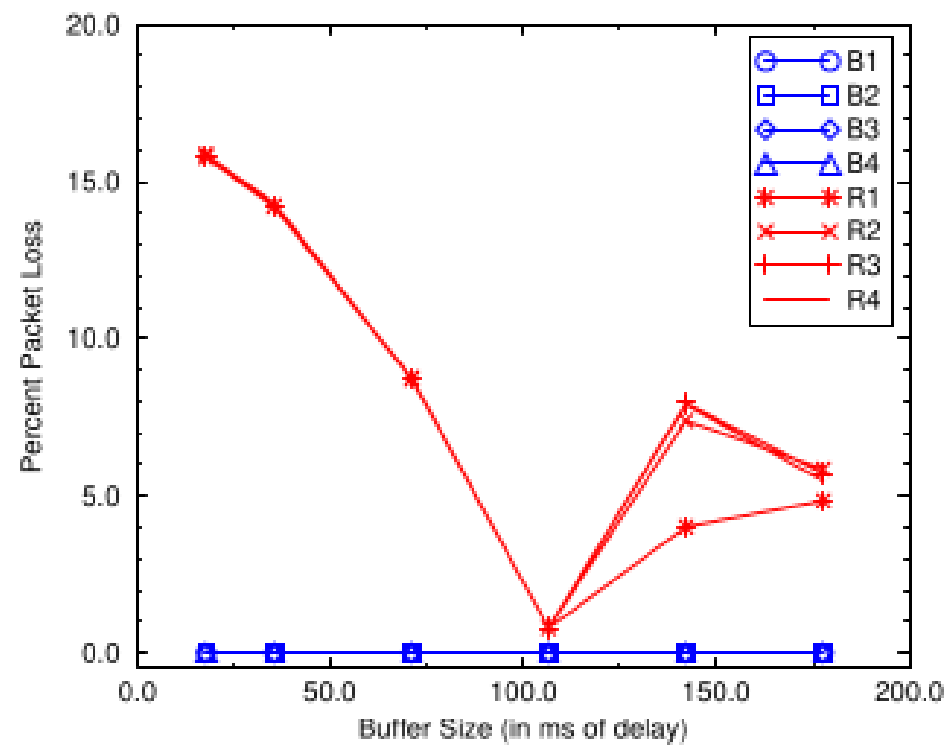
Configuration	w_q
$R1$	0.0002
$R2$	0.002
$R3$	0.02
$R4$	0.2

Table 1: RED configurations

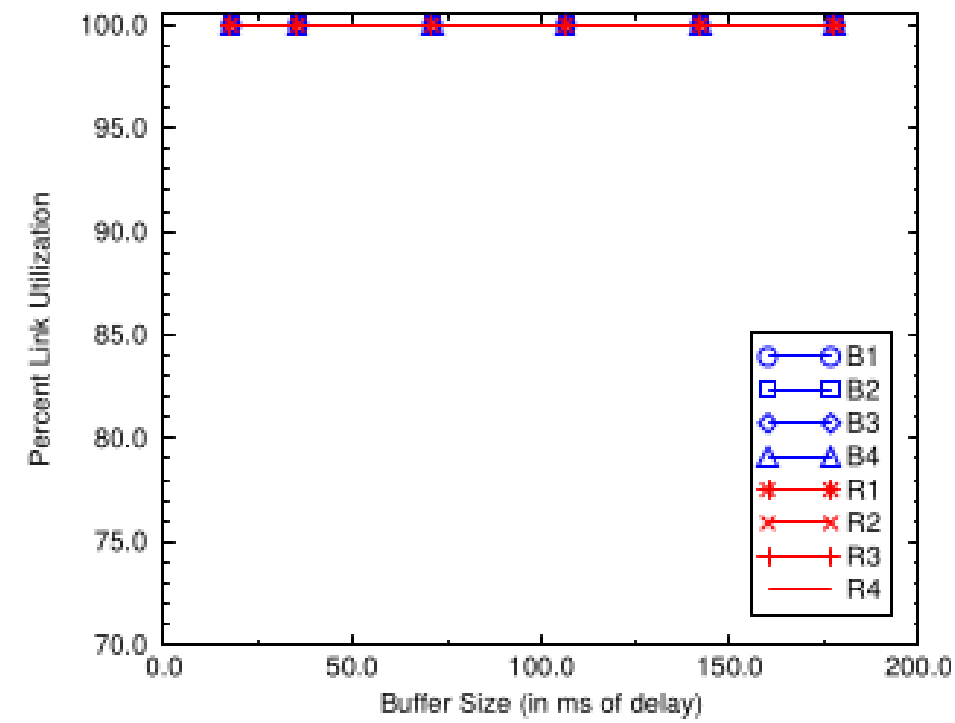
Configuration	$freeze_time$	$d1$	$d2$
$B1$	$10ms$	0.0025	0.00025
$B2$	$100ms$	0.0025	0.00025
$B3$	$10ms$	0.02	0.002
$B4$	$100ms$	0.02	0.002

Table 2: BLUE configurations

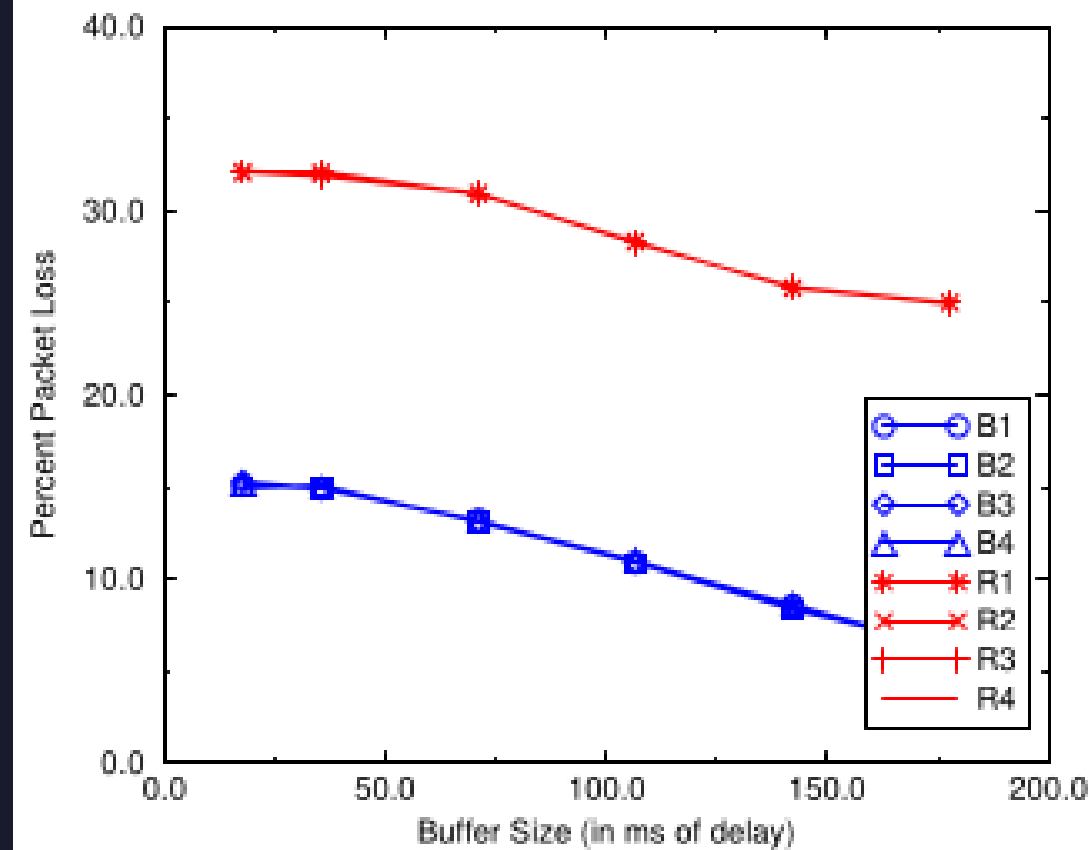
Experimental Results



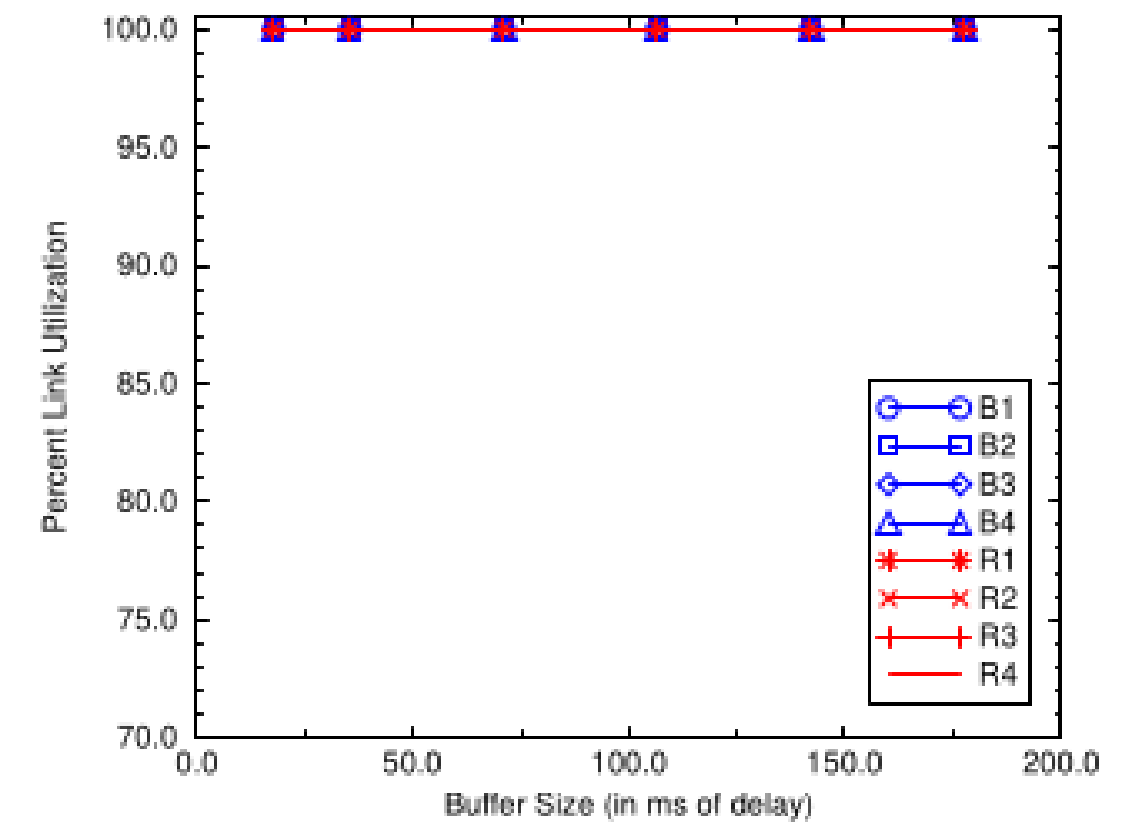
(a) Loss Rates (1000 sources)



(b) Link Utilization (1000 sources)

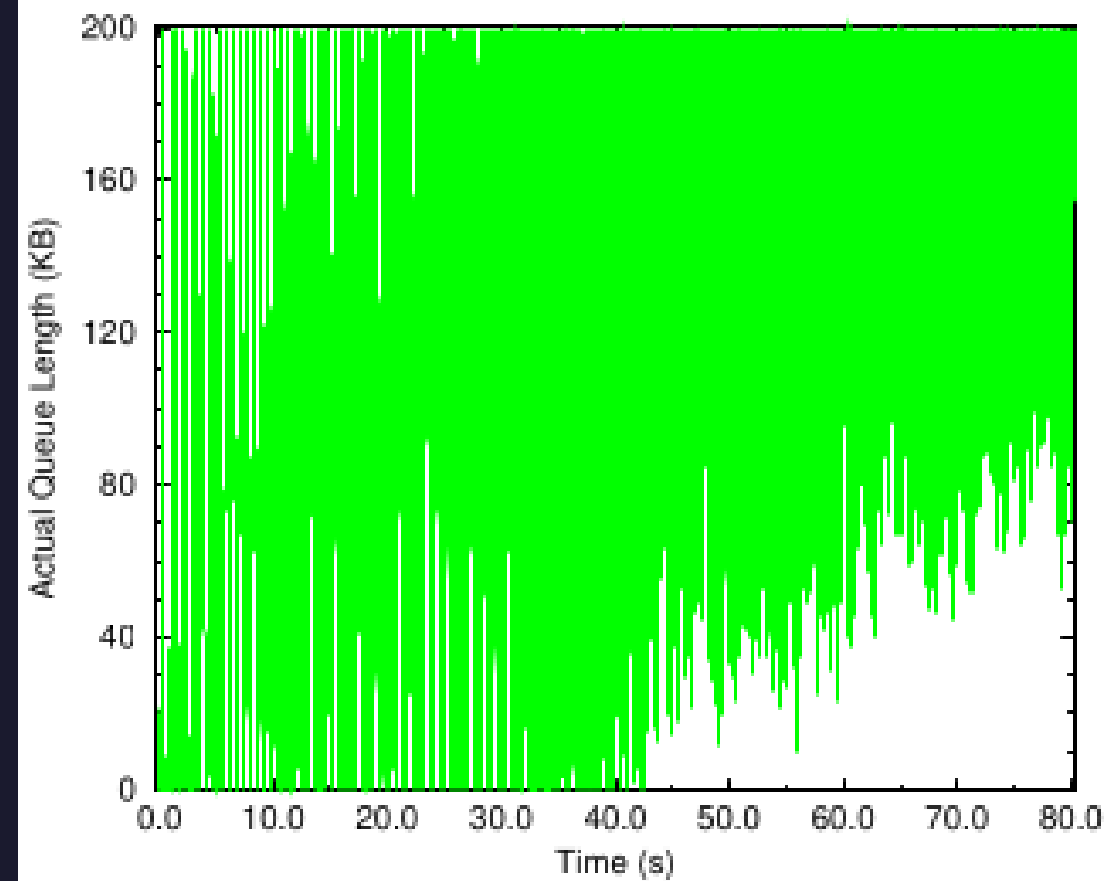


(c) Loss Rates (4000 sources)

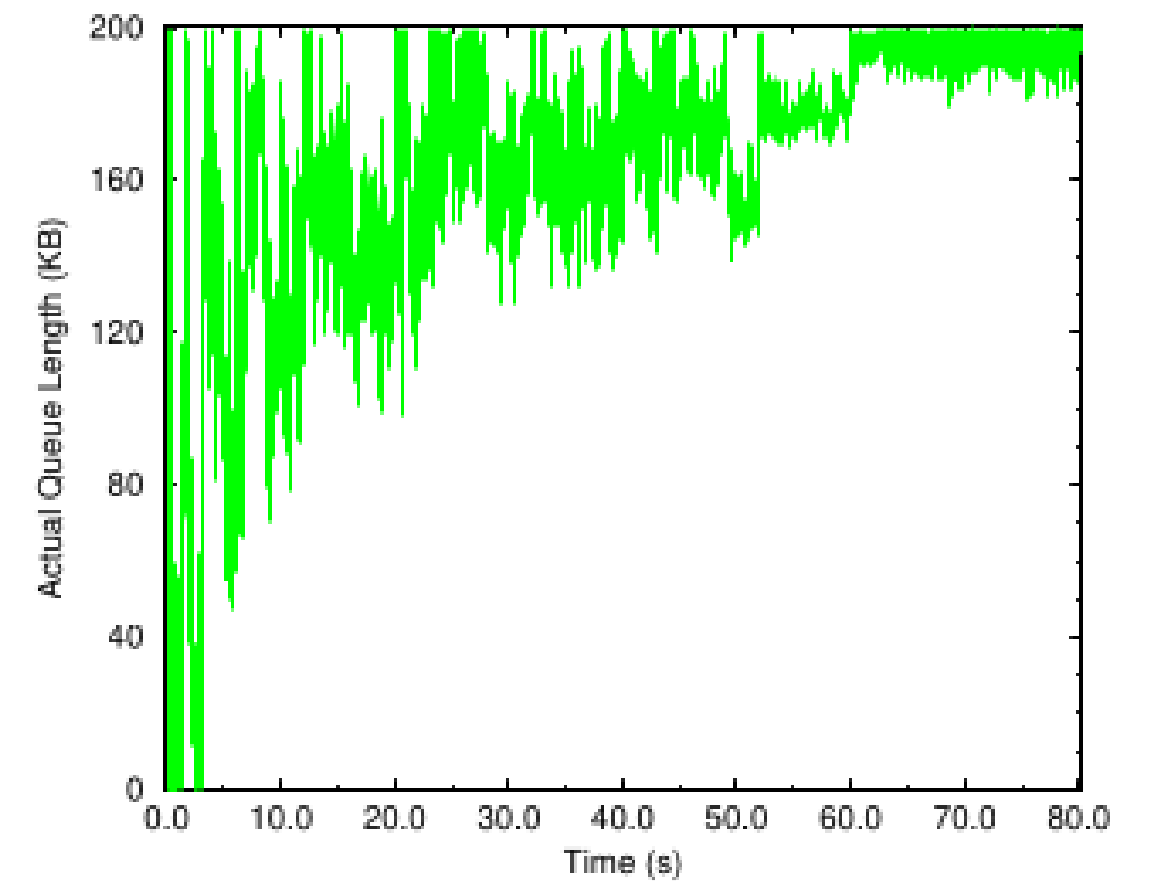


(d) Link Utilization (4000 sources)

Queue Length



(a) RED



(b) BLUE

Figure 6: Queue length plots of RED and BLUE

Marking Behaviour

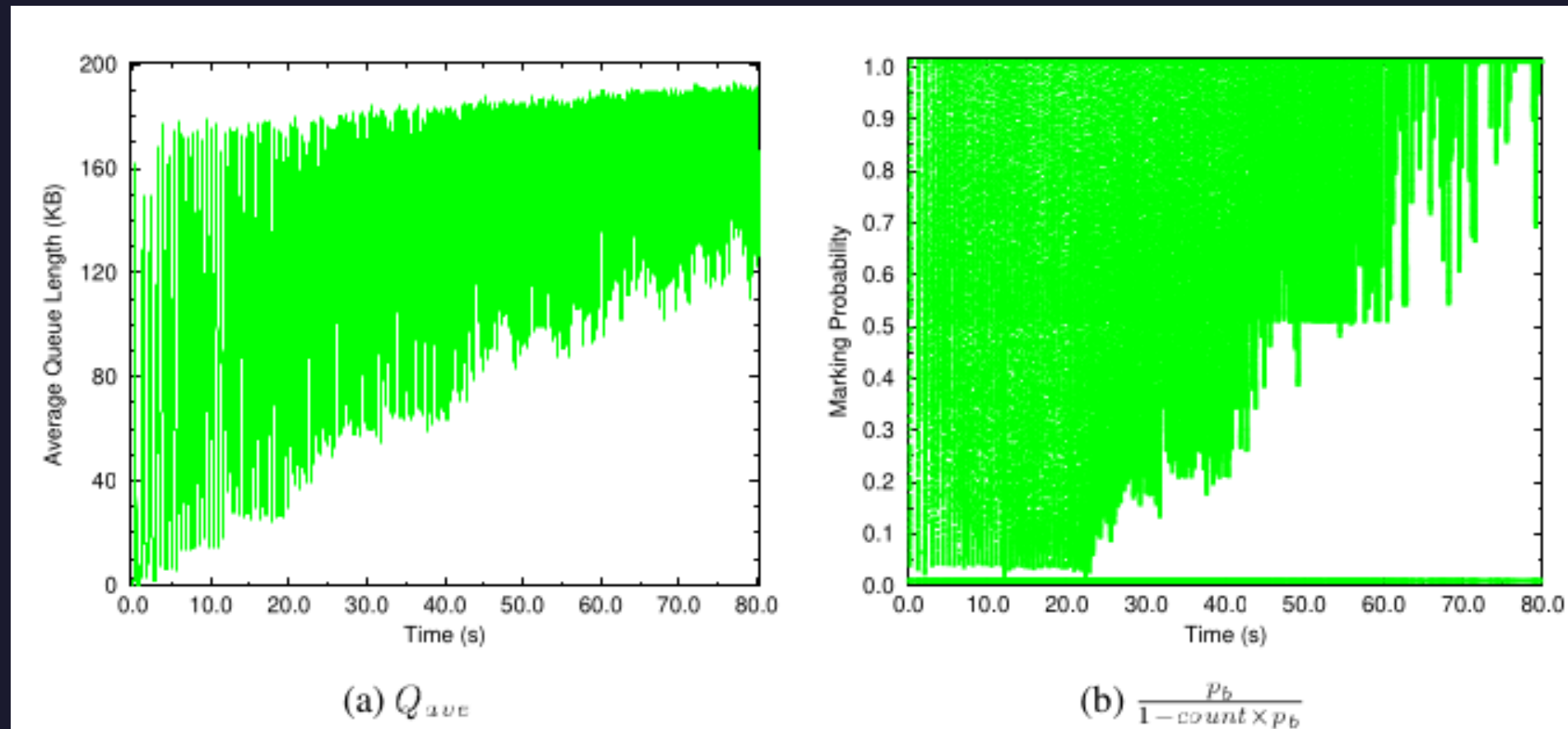


Figure 7: Marking behavior of RED

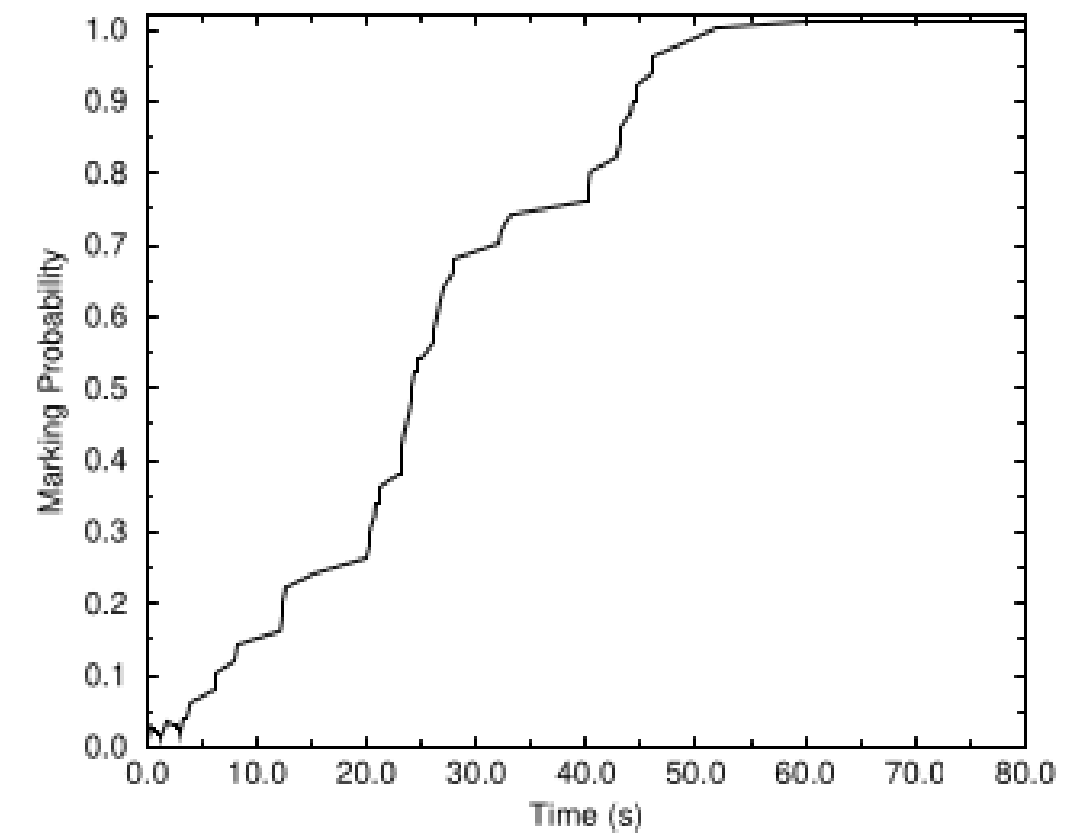


Figure 8: Marking behavior of BLUE (p_m)

Stochastic Fair BLUE (SFB) – Fair Congestion Control

The Problem SFB Solves:

- BLUE treats all flows equally
- Non-responsive flows (UDP streaming, malicious traffic) can monopolize bandwidth
- Responsive flows (TCP) suffer unfairly

SFB Algorithm :

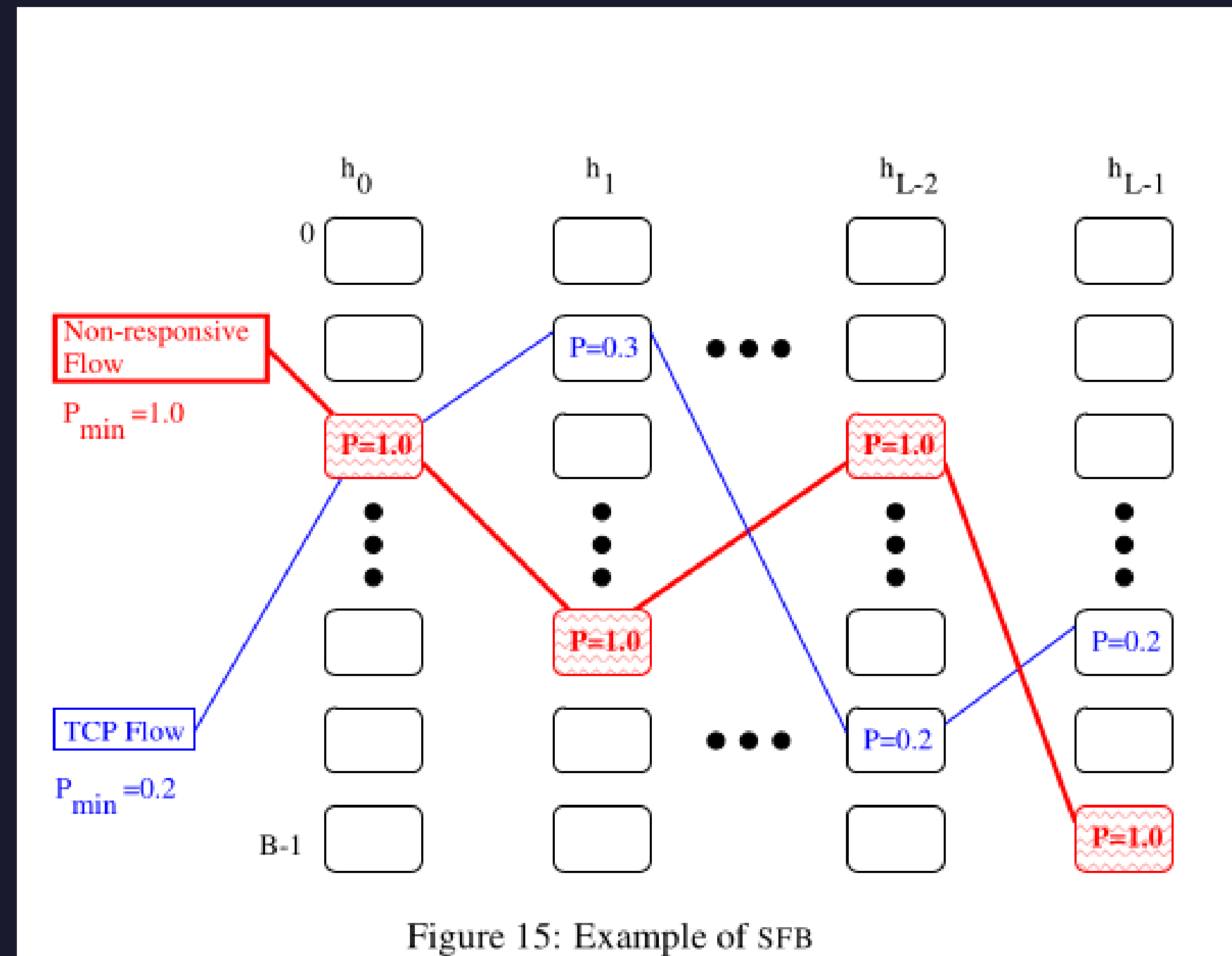
```
B[l][n]: L x N array of bins (L levels, N bins per level)
enqueue()
    Calculate hash function values  $h_0, h_1, \dots, h_{L-1}$ ;
    Update bins at each level
    for  $i = 0$  to  $L - 1$ 
        if ( $B[i][h_i].qlen > bin\_size$ )
             $B[i][h_i].p_m \mathrel{+}= \text{delta}$ ;
            Drop packet;
        else if ( $B[i][h_i].qlen == 0$ )
             $B[i][h_i].p_m \mathrel{-}= \text{delta}$ ;
     $p_{min} = \min(B[0][h_0].p_m \dots B[L][h_L].p_m)$ ;
    if ( $p_{min} == 1$ )
        ratelimit()
    else
        Mark/drop with probability  $p_{min}$ ;
```

Figure 14: SFB algorithm

Stochastic Fair BLUE (SFB) – Fair Congestion Control

How It Works:

- Each packet hashed to ONE bin per level (L bins total)
- Each bin maintains its own pm value
- Non-responsive flow → drives pm = 1 in ALL its bins
- TCP flow → likely has at least ONE clean bin (pm < 1)
- Final marking probability = minimum pm across all L bins



Research Gap – Why Modify BLUE?

1	BLUE works well for homogeneous TCP traffic
2	SFB provides fairness between flows
3	Modern networks have MIXED traffic (TCP + UDP + Real-time)
4	SFB has complexity: $O(L \times N)$ state, multiple hash functions
5	No adaptive mechanism for varying traffic conditions

Adaptive BLUE for Mixed Traffic

Core Idea

- Detect traffic type
- Adjust marking behavior
- Maintain fairness

Three-Component Architecture:

1. Traffic Classifier
2. Adaptive Marking
3. Dynamic Learning

Traffic Classifier:

```
if (packet.protocol == TCP)
    type = RESPONSIVE
else if (packet.protocol == UDP)
    if (flow_rate > threshold)
        type = NON_RESPONSIVE
    else
        type = REAL_TIME
```

Adaptive Marking

```
// TCP (Responsive)
pm_tcp = pm_base
```

```
// UDP Streaming (Non-responsive)
pm_udp = pm_base  $\times$   $\alpha$  (where  $\alpha > 1$ )
```

```
// Real-time (Gaming/VoIP)
pm_realtime = pm_base  $\times$   $\beta$  (where  $\beta < 1$ )
```

Dynamic Learning

```
On packet loss:
if (non_responsive_detected)
    pm = pm + (d1  $\times$   $\gamma$ )
else
    pm = pm + d1
On link idle:
pm = pm - d2
```


My Implementation

- 1 BLUE
- 2 Stochastic Fair Blue
- 3 Adaptive Blue
- 4 Compare Between them using Metrics(Fairness Index, Latency, Throughput, Complexity)

The background is a solid dark blue. It features two decorative elements made of thin white lines. On the left side, there is a series of concentric, wavy lines that curve upwards and then downwards, creating a sense of movement. On the right side, there is a similar series of concentric, wavy lines that curve downwards and then upwards, mirroring the design on the left. The text "Thank you" is centered in the middle of the image.

Thank you